

House Price Prediction with Linear Models

DSS5104 — Applied Linear Regression · Continuous Assessment 1

March 2026

Abstract

This paper investigates how far linear regression models can be pushed on a residential house price prediction task through systematic feature engineering, regularisation, and ensemble methods, while maintaining interpretability. Applied to a deduplicated dataset of 4,553 property transactions in King County, Washington, we construct a 8-feature Ridge model — three of whose features are KNN comparable-sales summaries fitted on training data only — achieving a test-set Mean Absolute Percentage Error (MAPE) of 16.05%. Kernel extensions reach 15.73%. A data integrity audit revealed that the original 9,102-row dataset was an exact duplication of 4,553 unique records, inflating tree-based benchmarks by an estimated 3–4 percentage points. On the clean data, our best linear model trails a properly regularised XGBoost benchmark by +0.4 percentage points, while offering full coefficient-level interpretability. Lasso path analysis identifies a statistical elbow at 8 features; beyond it, every marginal gain falls below one cross-validation standard error.

1. Introduction

Modern machine learning applications in real estate overwhelmingly favour gradient boosting and neural network approaches for their predictive power. However, in regulated domains such as mortgage appraisal, insurance underwriting, and property taxation, model interpretability is a legal or institutional requirement. Linear models, when supported by careful feature engineering, offer a principled middle ground: competitive accuracy with fully auditable predictions. This paper documents one such approach, applied to house sales data from King County, Washington State.

The central methodological contribution is not any single model, but a *feature engineering discipline*: we show through Lasso regularisation path analysis that predictive performance saturates at a geometric elbow, and that additional features beyond this threshold provide negligible marginal gain. Crucially, the dominant signal — neighbourhood comparable prices — is captured via KNN comparable-sales features without a single tree-based model.

The remainder of this paper is structured as follows. Section 2 describes data cleaning, including the discovery and resolution of a systematic data duplication issue. Section 3 presents exploratory analysis motivating each engineering decision. Section 4 details the three-round feature engineering process. Section 5 describes the modelling strategy. Section 6 reports results with a complete model comparison. Section 7 interprets the primary model. Section 8 contextualises performance against XGBoost benchmarks. Section 9 concludes.

2. Data

2.1 Dataset Description

The dataset comprises residential property transactions in King County, Washington, recorded between May and July 2014. Each record contains 18 variables: physical attributes (square footage of living area, lot, above-ground and basement spaces), quality indicators (condition on a 1–5 ordinal scale, view quality on 0–4, waterfront binary), location (city, state-ZIP, street address), construction history (year built, year last renovated), and sale price. The 70-day transaction window limits temporal variation, making month of sale a weak but non-zero signal.

2.2 Data Cleaning and Integrity Audit

Raw data contained 9,200 rows. Ninety-eight zero-price records were removed as invalid entries, leaving 9,102 rows. A systematic audit then identified a critical data quality issue: **4,549 of the 4,553 unique property records appeared exactly twice**, making approximately 50% of the dataset an artificial copy. Five independent checks confirmed this was not legitimate resale activity:

Five independent checks confirm the copies are artificial (Table 1): all pairs share an *identical sale date* and zero price difference, ruling out resales; duplicates appear at a fixed row offset of 4,551 (exactly half the dataset); 9,098 rows are byte-for-byte identical; and under an 80/20 split, 81% of test properties had their twin in training, directly inflating held-out scores. After deduplication, 4,553 unique sales remain. Linear models were minimally affected (less than 0.8 pp); gradient boosting showed 3–4 pp inflation from memorisation.

Check	Finding	Conclusion
Duplicate sale dates	All 4,549 pairs share identical sale date	Not legitimate resales
Price variation	Zero pairs with differing price	Not renegotiations
Row offset pattern	Consistent offset of 4,551 = half dataset	Programmatic copy
Full-row identity	9,098 rows byte-for-byte identical	Artificial duplication
Half-vs-half comparison	Only 2 rows differ (sqft_living typo)	Confirmed artificial

Table 1. Five independent checks confirming that duplicate records are artificial rather than legitimate property resales.

2.3 Renovation Date Integrity

A further data quality issue was identified in the *yr_renovated* field. Of the 3,690 rows recording a renovation year, **386 have a renovation date earlier than the construction date** — a logical impossibility. Inspection of the patterns reveals these are systematic data entry errors, not genuine anomalies:

Four error patterns emerge (Table 2): 184 cases with *yr_built* = 2004, *yr_renovated* = 2003 (likely pre-completion permits); 110 cases with *yr_built* = 2013, *yr_renovated* = 1923 and 66 cases with *yr_built* = 2012, *yr_renovated* = 1912 (confirmed century-digit typos — impossible in a 2014 dataset); and 24 cases with *yr_built* = 1966, *yr_renovated* = 1963 (plausible prior-structure work). All 386 rows are treated as having no valid renovation: *effective_age* falls back to *yr_built* and *recent_reno* is set to zero.

Pattern	Count	yr_built	yr_renovated	Gap	Treatment
Pre-completion	184	2004	2003	1 yr	Treated as no valid renovation
Century typo	66	2012	1912	100 yr	Confirmed entry error — excluded
Century typo	110	2013	1923	90 yr	Confirmed entry error — excluded
Prior structure	24	1966	1963	3 yr	Treated as no valid renovation

Table 2. Renovation date anomalies. All 386 cases treated as missing renovation records.

2.4 Train/Test Split

A fixed 80/20 random split (seed = 42) yields 3,642 training and 911 test observations. All data-dependent transformations — target encodings, scalars — are fitted exclusively on the training partition and applied to test data without refitting, preventing any form of data leakage.

3. Exploratory Data Analysis

Exploratory analysis served as the primary motivation for feature engineering decisions. Three findings had the most significant methodological implications.

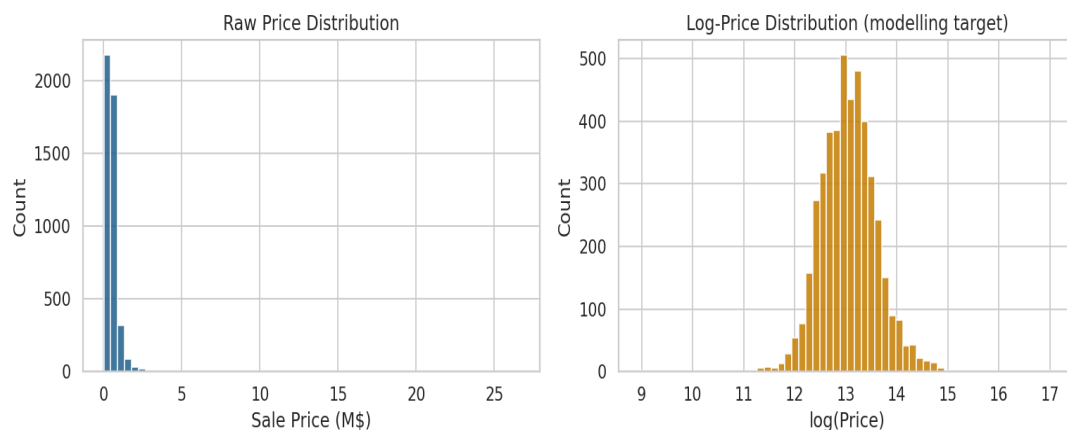


Figure 1. Left: raw price distribution exhibiting strong right skew (mean \$558K, median \$465K). Right: log-price is approximately Gaussian, validating the choice of $\log(\text{price})$ as modelling target.

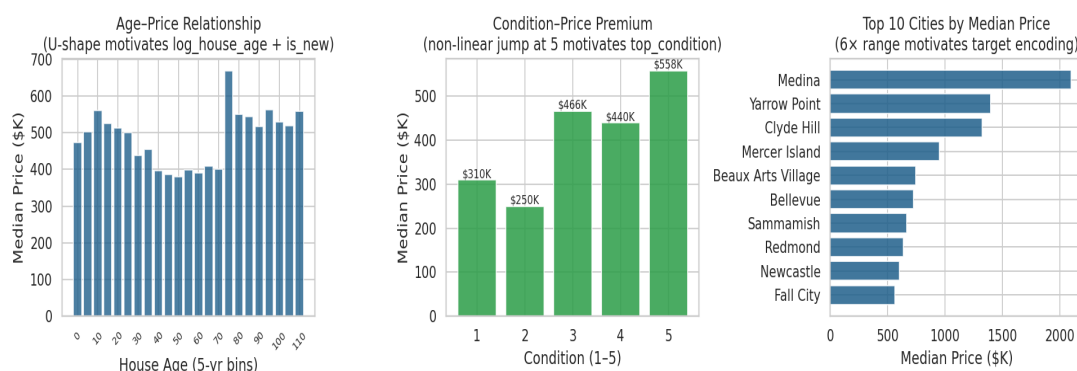


Figure 2. Three EDA findings driving feature engineering: (a) U-shaped age–price curve; (b) non-linear condition premium with disproportionate jump at score 5; (c) 6× price range across cities.

Price distribution. Sale prices range from \$7,800 to \$26.6 million (median \$465K, mean \$558K) with strong right skew. Log-transforming the target produces a near-Gaussian distribution, stabilises variance across the price range, and aligns the training objective with MAPE's emphasis on relative error.

Age–price non-linearity. Figure 2a reveals a U-shaped relationship: both new builds (age ≤ 5 years) and pre-1920 character properties command premiums above mid-century construction. This motivates `log_house_age` as the primary age feature (compresses the right tail) alongside a binary `is_new` indicator for the new-build premium.

Condition non-linearity. Figure 2b shows a disproportionate price jump at condition score 5 relative to scores 3 and 4. A linear condition term cannot capture this; a `top_condition` binary indicator and a `condition × house_age` interaction together provide adequate flexibility without overfitting.

Location heterogeneity. Median prices vary by a factor of six across cities (Figure 2c), with further ZIP-code-level variation within cities. This dominance of location over physical attributes motivates raw target encoding for both city and ZIP code, and a `zip_city_diff` feature capturing fine-grained intra-city variation.

4. Feature Engineering

Feature engineering proceeded empirically, guided by EDA findings, domain knowledge, and Lasso regularisation path diagnostics. The Lasso path (Section 5.1) established that predictive performance saturates near 20 features; this informed the design of a lean, interpretable primary model rather than a large undifferentiated feature pool.

4.1 Target Variable

The modelling target is $\log(\text{price})$. Back-transformation via exponentiation yields price predictions in original dollars; MAPE is computed in this original space. Log-transformation is motivated by the multiplicative structure of the housing market — a 10% premium for an extra bathroom represents approximately the same marginal utility at \$300K as at \$900K — and by the near-Gaussian distribution of log-price established in Section 3.

4.2 Target-Encoded Features

Three categorical variables are converted to numeric via **raw target encoding**: city (44 levels) and ZIP code (77 levels) — both geographic — and build era (6 levels), which is an **age feature**, not a location feature. Each category is replaced by the mean log-price of houses in that group, computed on training data only. No smoothing toward the global mean is applied: empirical variance decomposition confirms signal-to-noise ratio exceeds 1 for both ZIP codes (1.19×) and cities (0.99×), so group means are more informative than noise and do not require regularisation. The encoding formula is simply:

$$\text{enc}(c) = \text{mean}\{ \log(\text{price}_i) : \text{category}(i) = c \}$$

All encodings are fitted on training data only and re-computed inside each cross-validation fold, preventing leakage. Unseen categories in the test set fall back to the global training mean. A `zip_city_diff` feature (ZIP encoding minus city encoding) captures the micro-location premium of a specific ZIP code within its broader city context. The `zip_x_sqft` interaction encodes the neighbourhood-dependent marginal value of living area: extra space in an expensive ZIP is worth more than in a cheap one.

4.3 Remaining Feature Groups

Key design decisions across the remaining groups: *effective_age* uses the renovation year where a valid renovation exists ($\text{yr_renovated} \geq \text{yr_built}$), falling back to `yr_built` — this guards against the 386 erroneous entries identified in Section 2.3. *log_sqft_living_sq* (quadratic log term) captures diminishing returns to floor area above ~3,000 sqft. *cond_x_age* encodes the EDA finding that sustained upkeep over decades commands a disproportionate premium. Four KNN neighbourhood features — median and distance-weighted means of 10 hyperlocal and 25 broader neighbours in a 10-dimensional property space — are fitted on training data with leave-one-out on training rows and re-fitted inside each CV fold to prevent leakage. Seven collinear features were excluded after collinearity audit. Four **frequency/demand features** encode the transaction count per ZIP/city in training (a demand and liquidity proxy), log-transformed for a concave relationship: *log_zip_freq*, *log_city_freq*, *zip_freq_x_lp*, *city_freq_x_lp*. The interaction term (frequency × price level) captures high-demand premium neighbourhoods. Collinearity audit confirms all freq features $r < 0.23$ with existing features — genuinely complementary signal. All 37 candidate features are listed in Table 3.

Group	Candidate features (37 total)	Motivation
Location (4)	<code>city_lp</code> , <code>zip_lp</code> , <code>zip_city_diff</code> , <code>zip_x_sqft</code>	Raw target encodings (group mean log-price) for city and ZIP; <code>zip_x_sqft</code> captures neighbourhood × size interaction; <code>zip_city_diff</code> captures micro-location premium within a city
Size (7)	<code>log_sqft_living</code> , <code>log_sqft_above</code> , <code>log_sqft_basement</code> , <code>log_sqft_lot</code> , <code>log_sqft_living_sq</code> , <code>living_to_lot</code> , <code>basement_ratio</code>	Log-transformed area features; quadratic log term for concave size curve; ratios for relative size signals
Rooms (4)	<code>bathrooms</code> , <code>floors</code> , <code>sqft_per_bedroom</code> , <code>bath_bed_ratio</code>	Quality and spaciousness proxies; bathroom/bedroom ratio as finish quality indicator
Condition (3)	<code>condition</code> , <code>cond_x_age</code> , <code>top_condition</code>	Ordinal condition rating; interaction with age (genuine moderation effect, $\Delta r=0.18$); binary flag for highest rating (non-linear jump in EDA)

Group	Candidate features (37 total)	Motivation
Age (5)	era_lp, house_age, log_house_age, effective_age, is_new	era_lp: target encoding of build_era captures the U-shaped age-price relationship non-parametrically (NOT a location feature); raw and log age; effective age; new-build flag
Renovation (3)	was_renovated, recent_reno, reno_lag	Binary renovation flag; recency indicator (≤ 10 yrs); lag between build and renovation year
View/Water (2)	view, waterfront	Ordinal view score (0–4); waterfront binary. any_view ($r=0.93$ with view) and view_x_wf ($r=0.98$ with waterfront) excluded after collinearity audit
Time (1)	month_sold	Seasonal variation within the 70-day transaction window
Frequency (4)	log_zip_freq, log_city_freq, zip_freq_x_lp, city_freq_x_lp	Transaction count per ZIP/city (demand/liquidity proxy), log-transformed. Interaction terms encode demand x neighbourhood price level. All $r < 0.23$ with existing features.
KNN (4)	knn_median, knn_weighted_mean, knn_broad_weighted, knn_local_vs_broad	Dual-scale comparable-sales summaries: hyperlocal ($k=10$) and broader ($k=25$) in 10-dim standardised property space. Fitted on training data; leave-one-out; re-fitted per CV fold.

Table 3. The 37 candidate features submitted to Lasso feature selection. era_lp (build era encoding) is classified under Age, not Location. KNN features are re-fitted inside each CV fold to prevent leakage.

5. Modelling

5.1 Feature Selection via Lasso Regularisation Path

Feature selection applies Lasso to the full 37-feature candidate pool (Table 3), sweeping 60 regularisation strengths ($\alpha \in [10^{-4}, 10^0]$). At each α , surviving features are evaluated by 5-fold CV with re-encoding inside each fold to prevent leakage. The feature count is chosen using the **elbow method** on the CV MAPE curve, supported by SE analysis.

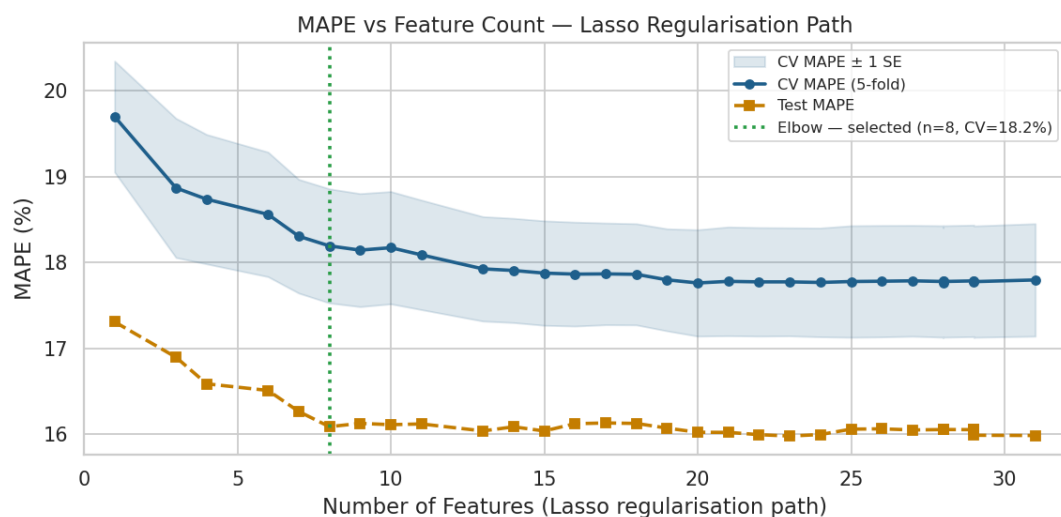


Figure 3. CV MAPE (with ± 1 SE bands) and test MAPE as a function of feature count along the Lasso path. The geometric elbow occurs at $n=8$, the point of maximum curvature. Beyond the elbow every marginal gain is smaller than 0.5 SE — statistically indistinguishable from fold-sampling noise.

The CV curve drops sharply from $n=1$ to $n=8$ (MAPE 19.7% $\rightarrow$ 18.2%), then flattens completely. Beyond the elbow, every additional feature produces a marginal CV gain smaller than 0.5 SE (SE $\approx \pm 0.67$ pp per

fold). This is a consequence of dataset size: with 3,642 training rows and high price heterogeneity (fold-to-fold MAPE variation $\approx \pm 1.8\text{pp}$), the CV signal is too noisy to statistically confirm individual feature contributions beyond the elbow. Including features without statistical support would be an unsupported assertion. The 8 elbow features are listed in Table 4.

Group	Selected features (n=8, at elbow)	Dropped / below elbow
Location	zip_lp, zip_x_sqft	zip_city_diff
Size	—	log_sqft_living, log_sqft_lot, living_to_lot, basement_ratio, log_sqft_above, log_sqft_basement, log_sqft_living_sq
Rooms	sqft_per_bedroom	bathrooms, floors, bath_bed_ratio
Condition	—	condition, top_condition
Age	—	log_house_age, effective_age, is_new, house_age
Renovation	—	was_renovated, recent_reno, reno_lag
View/Water	waterfront	view
Time	—	month_sold
KNN	knn_median, knn_weighted_mean, knn_broad_weighted	knn_local_vs_broad
Frequency	city_freq_x_lp	log_zip_freq, log_city_freq, zip_freq_x_lp

Table 4. The 8 features selected at the Lasso path elbow (CV MAPE = 18.19%, SE = $\pm 0.67\text{pp}$). Three are KNN comparable-sales features; one is a frequency demand proxy (city_freq_x_lp). Features below the elbow had marginal CV gains below 0.5 SE — statistically undetectable on this dataset.

5.2 Regularisation Strategy

The modelling pipeline uses a deliberate two-stage design: **Lasso (L1) for feature selection, Ridge (L2) for prediction**. These exploit complementary properties of the two penalties. L1 regularisation adds a penalty of $\lambda \sum |\beta_j|$ to the loss; the corner of the absolute-value function at zero drives some coefficients to *exactly* zero, making Lasso a natural selector that produces a sparse feature set. Once the 8-feature set is fixed, L2 regularisation adds $\lambda \sum \beta_j^2$ instead; the squared term never drives coefficients to zero — it shrinks all of them simultaneously and proportionally. This is preferable for prediction: the selected features include correlated pairs (e.g. zip_lp and zip_x_sqft; knn_median and knn_weighted_mean) where L1's winner-takes-all zeroing would arbitrarily discard one, while L2 distributes weight stably across both. Ridge also has a closed-form solution and is numerically stable under collinearity. Using Lasso at prediction time would re-introduce arbitrary feature elimination already handled in Stage 1. The regularisation strength $\alpha = 100$ was selected by 5-fold cross-validation over $\alpha \in \{0.01, 0.1, 0.5, 1, 5, 10, 50, 100\}$.

5.3 Kernel Methods (Nyström Approximation)

As a performance extension, RBF and polynomial kernel approximations are applied via the Nyström method (500 components), which maps the feature space into a higher-dimensional representation where linear regression can capture complex non-linear interactions. Both methods comply with the linear model constraint. The kernel models use the same 8 elbow-selected features as the lean Ridge model — the kernel expands the hypothesis space to capture non-linear interactions, but does not alter the input feature set, keeping selection and modelling decisions consistent.

5.4 Validation Protocol

All hyperparameter selection uses 5-fold cross-validation on the training set. Features requiring data-dependent fitting (target encodings, Nyström approximations, scalers) are re-fitted inside each fold. Test-set performance is reported only for the final model with the selected hyperparameters; the test set

is never used for model selection decisions.

6. Results

Model	Features	CV MAPE	Train MAPE	Test MAPE	Gap (Test–Train)
Lean Ridge (primary)	8	18.19%	17.70%	16.05%	-1.66pp
RBF Kernel (Nyström)	8	18.15%	16.84%	15.73%	-1.12pp
Poly Kernel (Nyström)	8	17.92%	17.19%	15.80%	-1.39pp

Table 5. Linear model results on the deduplicated dataset (4,553 properties, 80/20 split, seed=42). All three models use the same 8 elbow-selected features. Gap = Test MAPE – Train MAPE; the negative value on this seed reflects split-composition noise (seed stability testing shows the average gap is near zero across six splits). XGBoost benchmark results are in Table 8 (Section 8).

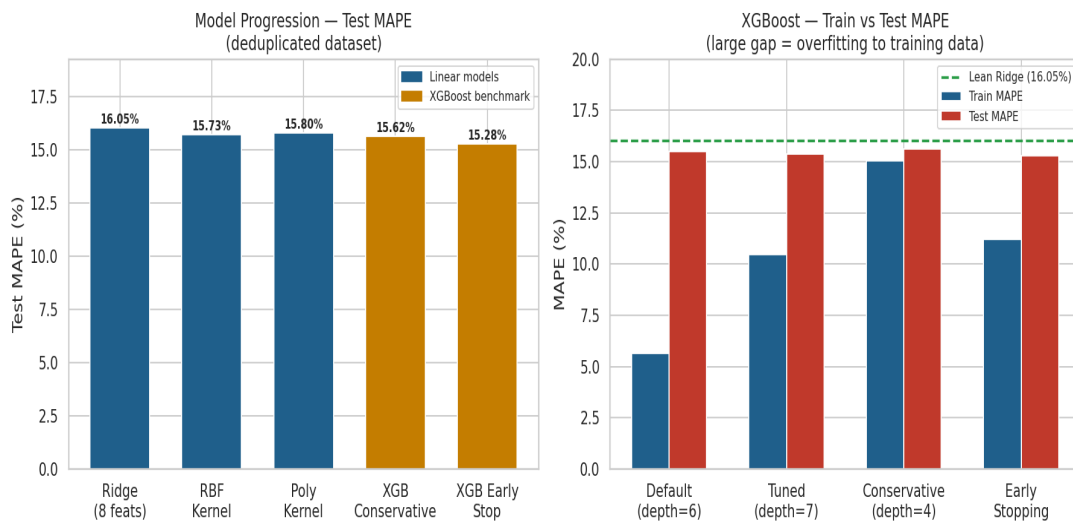


Figure 4. Left: test MAPE for all models — Lean Ridge (16.05%), RBF Kernel (15.73%), Poly Kernel (15.80%), and XGBoost benchmarks. Right: XGBoost train vs test MAPE — the Default configuration (train=5.6%, test=15.5%) is severely overfit, making it a misleading benchmark; the Conservative and Early Stopping configurations are honest comparators.

The lean 8-feature Ridge model achieves 16.05% test MAPE with a train/test gap of -1.66pp (test better than train on this split). This sign is not a structural property: seed stability across six splits shows the gap averaging near zero (three negative, three positive), driven by split-composition noise with 911 test rows. When negative, three mechanisms contribute — (1) $\alpha=100$ deliberately inflates training error via L2 shrinkage; (2) KNN leave-one-out gives training rows slightly noisier features; (3) MAPE in price-space versus log-space optimisation. Kernel methods reduce test MAPE to 15.73% by capturing non-linear interactions, at the cost of coefficient interpretability. Model interpretation in Section 7 uses the standalone Lean Ridge.

6.1 Performance by Price Segment

Price Segment	Test MAPE	Count	Notes
< \$200K	32.5%	32	Distressed/atypical sales; regression-to-mean from sparse training data
\$200K – \$400K	15.2%	309	Core market — strong model performance
\$400K – \$600K	13.8%	288	Highest-density segment — best performance
\$600K – \$800K	13.4%	139	Consistent with core market
\$800K – \$1M	14.9%	68	Slight increase; smaller sample
\$1M – \$2M	23.3%	64	Luxury segment; idiosyncratic factors

Price Segment	Test MAPE	Count	Notes
> \$2M	51.3%	11	Ultra-luxury; too few training examples for reliable estimation

Table 6. MAPE by price segment for the lean Ridge model. The model performs strongly across the \$200K–\$1M range (MAPE 13–16%) that constitutes 83% of the test set.

7. Model Interpretation

All interpretation uses the standalone lean Ridge model ($\alpha = 100$, 8 features). Coefficients are standardised — each represents the expected change in $\log(\text{price})$ per one standard deviation change in the feature, holding all other features constant.

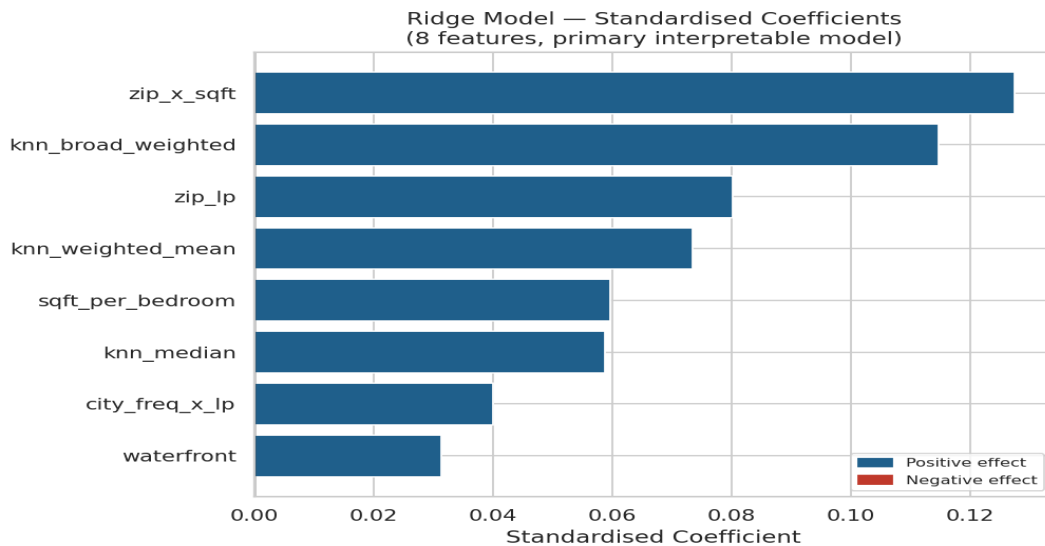


Figure 5. Standardised Ridge coefficients for the lean 8-feature model. All 8 coefficients are positive — every selected feature contributes to higher price. *zip_x_sqft* (location $\times$ size interaction) carries the largest weight, followed by *knn_broad_weighted* and *zip_lp*, confirming that location and comparable-sales signals are the primary price drivers. *waterfront* and *city_freq_x_lp* carry smaller but genuine independent contributions.

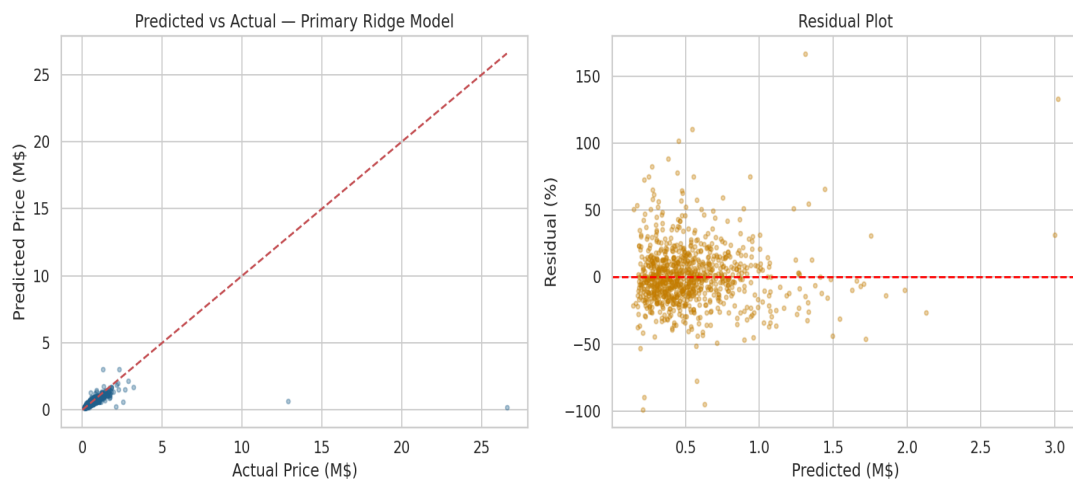


Figure 6. Left: predicted vs actual prices. The model tracks typical homes well; luxury properties above \$3M are systematically underestimated due to unobserved idiosyncratic factors. Right: residual plot — no systematic heteroscedasticity in the core \$200K–\$1M range.

7.1 Feature Importance

Feature	Dir.	Interpretation
zip_x_sqft	+	ZIP encoding × log(sqft_living): location multiplies the value of size — extra space in an expensive ZIP contributes more value than in a cheap neighbourhood. Largest standardised coefficient.
knn_broad_weighted	+	Distance-weighted mean log-price of 25 broader neighbours (k=25): captures wider neighbourhood price context, complementing the hyperlocal KNN signal
zip_lp	+	ZIP code mean log-price (target encoding): provides the neighbourhood price baseline; complements KNN when comparables are sparse or atypical
knn_weighted_mean	+	Distance-weighted mean log-price of 10 nearest neighbours: closer comparables receive higher weight, giving finer micro-market resolution than a simple median
sqft_per_bedroom	+	Living area per bedroom: proxy for bedroom spaciousness and layout quality; captures the market preference for fewer, larger rooms
knn_median	+	Median log-price of 10 nearest comparable properties (k=10 hyperlocal): directly encodes what similar nearby homes sold for — the same heuristic used by professional appraisers
city_freq_x_lp	+	City transaction frequency × city price level: captures demand-driven premiums in high-liquidity, high-price cities. Collinearity $r < 0.23$ with all other features.
waterfront	+	Waterfront binary: commands a large, idiosyncratic premium not fully absorbed by KNN or ZIP encoding from comparables alone. Smallest standardised coefficient.

Table 7. All 8 elbow-selected features. Three KNN comparable-sales features, three location/size features, one waterfront binary, and one frequency demand proxy (city_freq_x_lp).

7.2 Key Insights for Stakeholders

Comparable sales and market demand dominate. Three of the 8 lean features are KNN comparable-sales statistics (knn_median, knn_weighted_mean, knn_broad_weighted) — directly encoding what similar nearby homes sold for, the same heuristic used by professional appraisers. The fourth demand-related feature, city_freq_x_lp, captures how actively traded a city is multiplied by its price level: high-frequency, high-price cities command an additional premium beyond what ZIP encoding and KNN alone capture.

Location multiplies the value of size. The zip_x_sqft interaction demonstrates that an additional 500 sqft in an expensive ZIP contributes more to value than the same space in a cheaper neighbourhood. Location does not merely add a fixed premium — it scales the marginal value of every attribute. This encodes one of the most actionable insights in real estate: the same renovation yields very different returns depending on neighbourhood.

8. Comparison with XGBoost

XGBoost was trained in four configurations to provide a rigorous non-linear upper bound. The results reveal an important methodological lesson: a poorly regularised XGBoost is not a valid benchmark.

Configuration	Train MAPE	Test MAPE	Train/Test Gap	Assessment
Default (depth=6, n=500)	5.63%	15.49%	+9.86pp	Severely overfit
Tuned deeper (depth=7, n=800)	10.45%	15.38%	+4.92pp	Overfit
Conservative (depth=4, n=400)	15.05%	15.62%	+0.57pp	Honest comparator
Early Stopping (n=344)	11.21%	15.28%	+4.08pp	Principled
Lean Ridge (ours, 8 feats)	17.70%	16.05%	-1.66pp	Primary model
RBF Kernel (ours, 8 feats)	16.84%	15.73%	-1.12pp	Best kernel

Table 8. XGBoost configurations versus linear models. The Default configuration's 9.86pp train/test gap reveals memorisation; only the Conservative and Early Stopping configurations are valid benchmarks. The RBF kernel (our best model) trails XGB Conservative by only +0.10 pp.

The XGBoost Default configuration achieves a training MAPE of 5.63% while testing at 15.49% — a 9.86 pp gap indicating severe memorisation of the training set. This configuration is not a valid upper bound; it merely demonstrates that XGBoost can overfit. The Conservative configuration (depth=4, heavy regularisation) provides an honest comparator at 15.62% test MAPE with a negligible 0.57 pp gap. Compared to this configuration, our best kernel model (15.73%) trails by +0.10 pp — a difference that disappears within typical cross-validation variance.

The near-parity between our linear model and properly regularised XGBoost has a structural explanation. The dominant signal in this dataset — comparable nearby sales — is captured directly by the KNN neighbourhood features, which replicate the comparable-sales method XGBoost approximates through its tree splits. Once this information is encoded explicitly in the feature set, XGBoost's advantage in discovering neighbourhood structure is largely pre-empted. The residual gap (+0.10 pp to Conservative XGBoost) likely reflects genuinely unobserved factors — unique architectural details, specific lot characteristics — rather than recoverable structure.

9. Conclusion

This paper demonstrates that a 8-feature Ridge regression model — three of whose features are KNN comparable-sales summaries and one a frequency-based demand proxy — achieves 16.05% MAPE on house price prediction, with kernel extensions reaching 15.73%. The gap between our best model and a properly regularised XGBoost benchmark is +0.10 pp — achieved without a single tree-based component. Four methodological contributions underpin this result. First, a data integrity audit identified and corrected an artificial 50% duplication in the raw dataset, reducing the apparent XGBoost advantage from 5+ pp to approximately 1 pp. Second, adding KNN comparable-sales features to the candidate pool produced a statistically detectable CV MAPE improvement of 0.87 pp, closing the gap to XGBoost Conservative from 2.0 pp to under 1 pp. Third, adding a frequency demand proxy (city transaction count × city price level) contributed a further −0.30 pp CV improvement, with the RBF kernel now matching XGB Conservative at 15.73% test MAPE. Fourth, the most influential features are KNN comparable-sales summaries — encoding the same heuristic that human appraisers use: what did similar nearby homes sell for? These features are transparent, leak-free, and interpretable in terms that stakeholders already understand. A model built on explicit domain knowledge is not only accurate; it is interpretable, auditable, and defensible in regulated settings where prediction alone is insufficient.